Hands-On Generative AI

Adapting Pre-trained Models

Day 3 - Session 1

KavinSchool

Where We Left Off: Day 2 Recap

Yesterday we moved from raw text to models that read, generate, and act.

- **Session 1:** NLP as data: tokenization, vectorization, embeddings
- **Session 2:** Pre-trained models: BERT vs GPT and the Transformer
- **Session 3:** Language generation tasks and prompting techniques
- **Session 4:** Agentic AI: tool use, function calling, ReAct, MCP

Today we adapt those pre-trained models to a specific task.

What We'll Cover

1. Transfer learning: reuse instead of train from scratch
2. Feature extraction versus full fine-tuning
3. Demo: feature extraction with a frozen encoder
4. Cost and catastrophic forgetting
5. Demo: catastrophic forgetting
6. Fine-tuning with Hugging Face transformers
7. Lab 3.1: fine-tune DistilBERT for sentiment

Transfer Learning

Reuse what a big model already learned

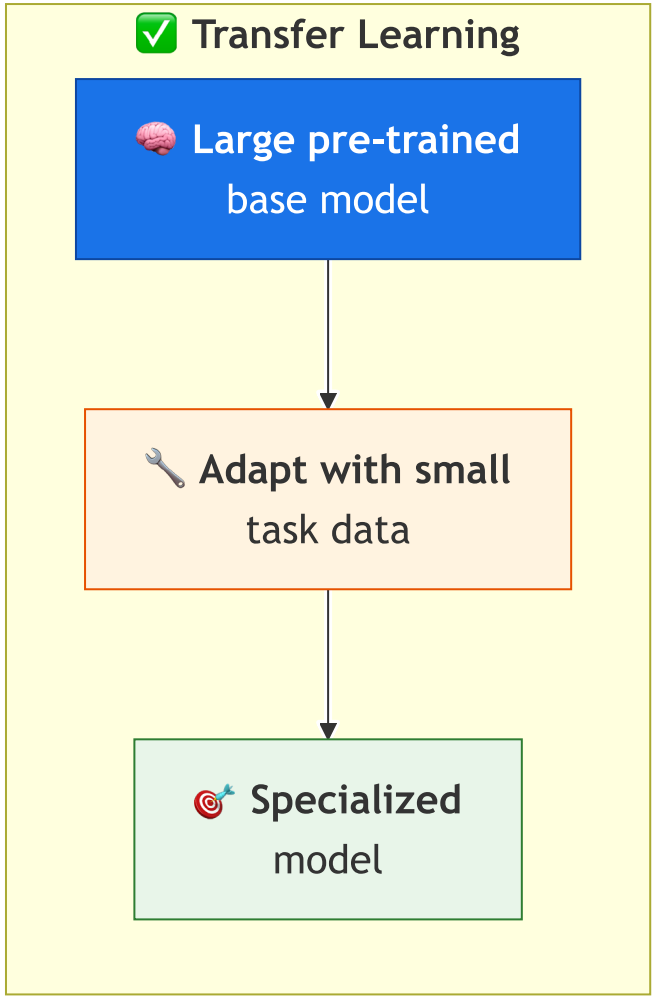
KavinSchool

Why Not Train From Scratch

Training a model from scratch needs a huge dataset and a large compute budget. Most teams cannot afford that, and they do not need to.

- A pre-trained base already encodes general features
- You adapt it with a small amount of task data
- You reach good accuracy with less data and time

Transfer Learning in One Picture



What Transfer Learning Gives You

Start from a base model trained on broad data, then specialize it for your task.

- Less labeled data needed
- Faster training, lower cost
- Reuse of features the base already learned

The base does the heavy lifting; your data shapes the last mile.

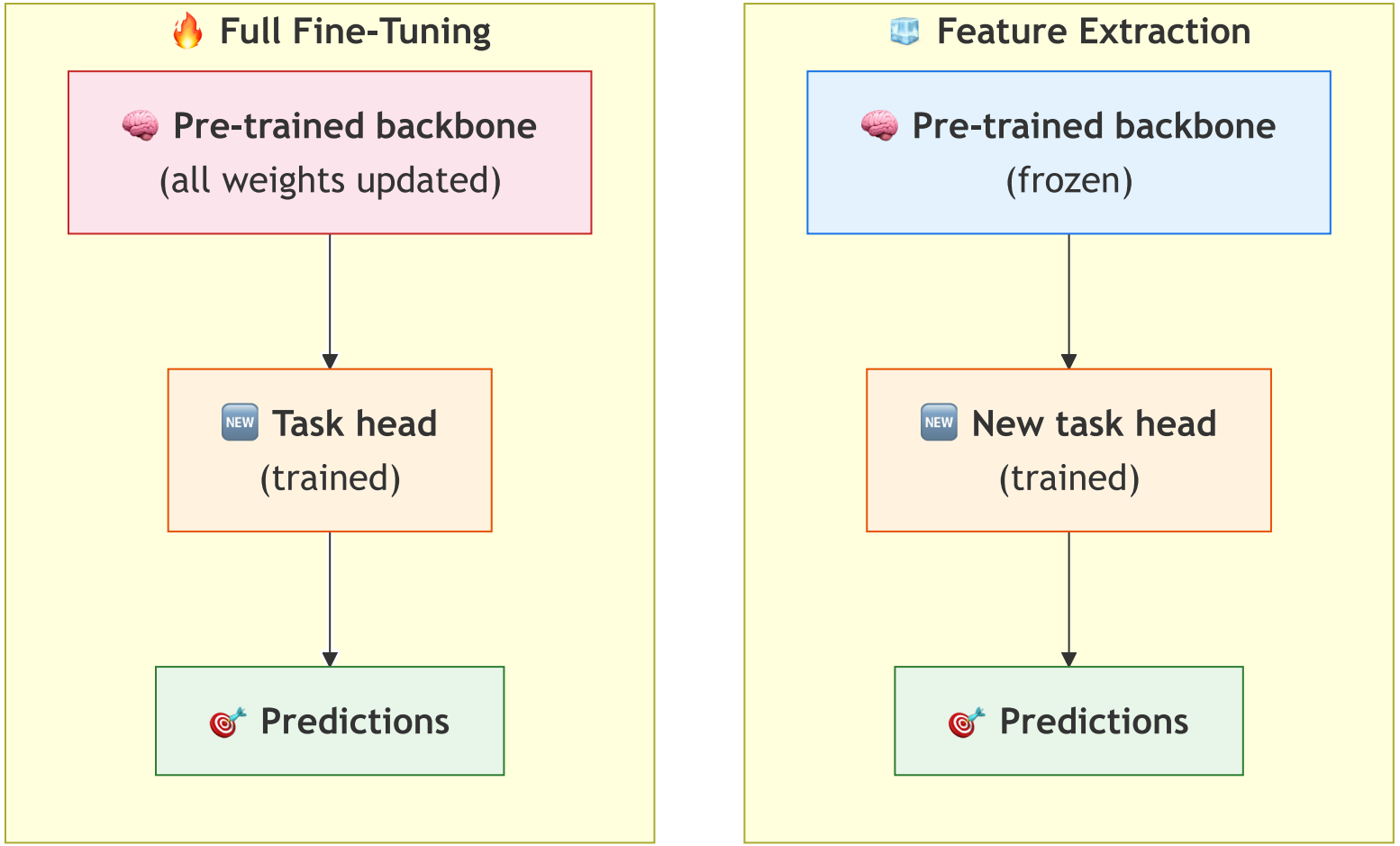
> **IN PRACTICE**
>
> Customer support teams start with a base model (like Llama or Mistral) and fine-tune it on their specific resolution ticket histories. The base knows how to write; the fine-tune knows company policy.

Two Ways to Adapt

Feature extraction versus full fine-tuning

KavinSchool

Feature Extraction vs Full Fine-Tuning



Comparing the Two

Aspect	Feature extraction	Full fine-tuning
Weights updated	New head only	All weights
Compute	Low	High
Data needed	Small	Larger
Overfitting risk	Low	Higher

Feature extraction freezes the backbone. Full fine-tuning updates everything.

Demo: Feature Extraction

Run `day3/demos/demo-feature-extraction.py`.

- Embed sentences with a frozen sentence-transformers encoder
- Train only a small logistic-regression head (385 parameters)
- Classify unseen sentences without touching the base weights

> A frozen base plus a tiny head learns a new task cheaply.

When to Fine-Tune

Try the cheaper options first and fine-tune only when they fall short.

- **Prompting:** no training, immediate, cheap to iterate
- **RAG (retrieval-augmented generation):** private or changing facts (Session 3)
- **Fine-tuning:** fixed style or format, stable labeled data

Fine-tune when you have labeled examples and the target behavior does not change often.

> **TIP**
>
> Always build a robust prompting baseline first. You cannot know if fine-tuning is worth the cost until you know what a few-shot prompt achieves.

Cost and Forgetting

The two things that bite you

KavinSchool

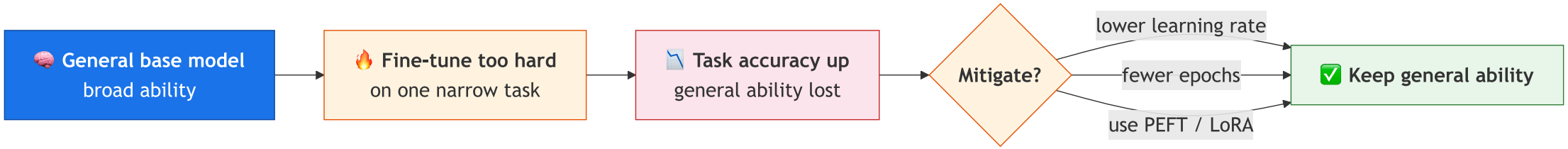
The Cost of Full Fine-Tuning

Full fine-tuning of a large model is expensive on two fronts.

- **Memory:** GPUs must hold weights, gradients, and optimizer state
- **Storage:** each fine-tuned model is a full-size checkpoint

These costs are why parameter-efficient methods exist. We cover those next session.

Catastrophic Forgetting



Avoiding Catastrophic Forgetting

Train too hard on a narrow task and the model loses general ability while task accuracy climbs. Keep the changes gentle.

- Use a lower learning rate
- Train for fewer epochs and watch a validation set
- Prefer parameter-efficient fine-tuning (Session 2)
- Mix in some general examples

> **WARNING**
>
> If you train exclusively on short JSON responses, the model may permanently lose its ability to hold a conversational dialogue.

Demo: Catastrophic Forgetting

Run `day3/demos/demo-catastrophic-forgetting.py`.

- Train one small net on Task A until accuracy hits 1.00
- Train the same net on Task B only
- Watch Task A accuracy collapse as Task B accuracy climbs

> Narrow, hard fine-tuning overwrites earlier skills.

Fine-Tuning in Practice

Hugging Face transformers

KavinSchool

The Trainer Workflow

The `transformers` library handles the training loop, batching, and evaluation.

- Load a pre-trained model and its tokenizer
- Tokenize your labeled dataset
- Set training arguments (epochs, batch size, learning rate)
- Call `trainer.train()`

The same shape works across tasks by swapping the model class and dataset.

Trainer in Code

```
model = AutoModelForSequenceClassification.from_pretrained(
    "distilbert-base-uncased", num_labels=2)

args = TrainingArguments(output_dir="out", num_train_epochs=1,
                         per_device_train_batch_size=8,
                         learning_rate=2e-5)
trainer = Trainer(model=model, args=args, train_dataset=train_ds)
trainer.train()
```

A short run on a small dataset is enough to see the model shift.

Summarization as a Fine-Tune

Summarization uses a sequence-to-sequence model such as T5 or BART. The workflow matches classification: load the model, tokenize document and summary pairs, train.

- Swap in `AutoModelForSeq2SeqLM`
- Prompting a capable model handles most summarization already
- Fine-tune only for a specific summary style or length

Intuit Scenario: QuickBooks Transaction Categorization

Fine-tuning adapts general knowledge to domain-specific patterns.

- **Base model:** Understands general English text and finance concepts
- **Task:** Classify raw bank feed text into accounting charts of accounts
- **Dataset:** Thousands of historical mapped transactions per industry
- **Outcome:** A small, fast model tuned exactly for bookkeeping nuances, running locally

Breakout Lab 3.1

Fine-tune DistilBERT for sentiment

KavinSchool

Lab 3.1: Fine-tune DistilBERT

Open `day3/lab3_1_bert_finetune/start.ipynb` in the companion repo.

- Load `distilbert-base-uncased` (a smaller, faster BERT) and a tiny labeled sentiment subset
- Run a few training steps with the Trainer
- Compare predictions before and after
- Note how a small adaptation shifts the outputs

> Stretch goal: read the summarization notes and sketch the seq2seq changes.

Key Takeaways

- [x] Transfer learning reuses a pre-trained base instead of training from scratch
- [x] Feature extraction freezes the backbone; full fine-tuning updates all weights
- [x] Full fine-tuning costs memory and storage per task
- [x] Fine-tuning too hard causes catastrophic forgetting; keep changes gentle
- [x] Hugging Face Trainer runs the fine-tuning loop for you

Questions?

Next: Parameter-Efficient Fine-Tuning and Sampling

Short break, then Session 2

KavinSchool